Stock Price Movement Prediction from Financial News

Fine-Tuning ALBERT for Binary Headline Classification

Santiago Freile · April 2026

Overview

This report describes the design, training, and evaluation of a binary classifier that predicts stock price direction – UP or DOWN – based on a financial news headline. The model is built on ALBERT (A Lite BERT), a lightweight transformer architecture developed by Google, fine-tuned on a synthetically generated dataset of 1,600 labeled headlines.

The most informative result is not the 100% accuracy on the test set. It is what happens when the model is tested on plain English headlines it was never trained on, a set of behavioral probing experiments that reveal exactly what the model learned and where it fails.

Dataset

The dataset consists of 1,600 financial news headlines, split evenly between two classes: Stock UP (label 1) and Stock DOWN (label 0). The data was generated synthetically using templated language patterns typical of financial news reporting.

Class	Count	Proportion
Stock UP (1)	800	50%
Stock DOWN (0)	800	50%
Total	1,600	100%

Table 1: Class distribution. Perfect 50/50 balance.

Linguistic Patterns

Manual inspection of the headlines reveals clear vocabulary differences between the two classes. Bullish headlines tend to use words like *strong, lifting, confidence, gains, announces.* Bearish headlines lean on *decline, concerns, slowing, hurting, faces, raising.*

However, a number of words appear frequently in both classes. General financial vocabulary like *trading, shares, quarter, reports*, which carry no directional signal on their own. The word *shares*, for example, appears in both:

- *"shares jump after strong demand"* → UP
- *"sending shares lower"* → DOWN

This confirms that a bag-of-words model would fail in this task. Sentiment lives in the combination and context of words, not in individual terms. A contextual language model like ALBERT, which reads the entire sequence simultaneously, is better suited.

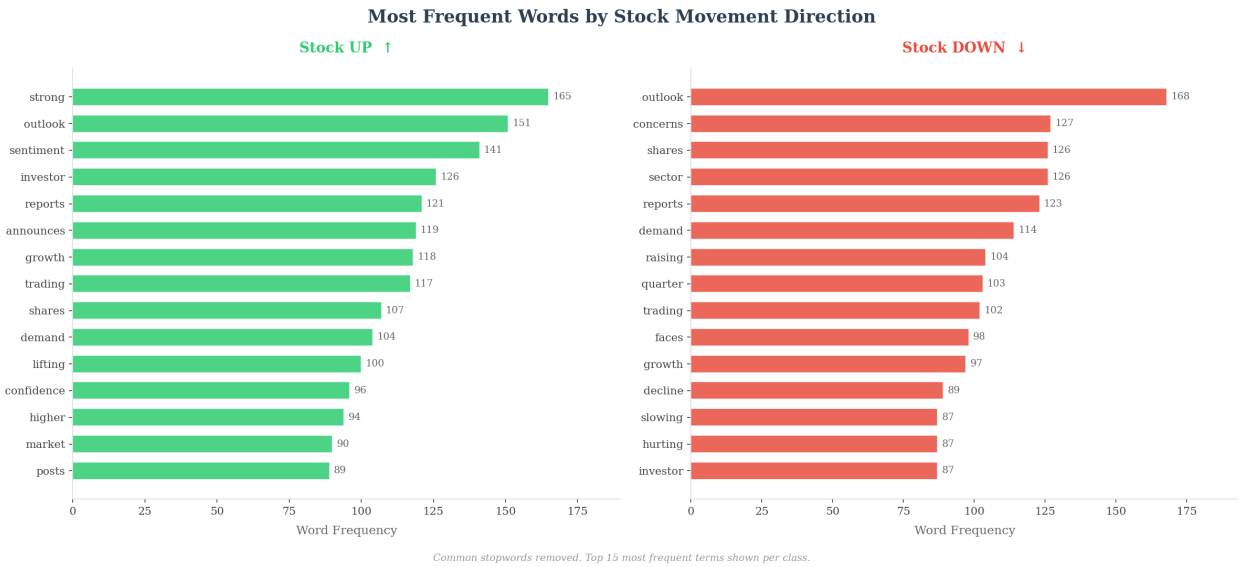

Figure 1: Top 15 most frequent words per class after stopword removal.

Headline Length

Headlines range from 8 to 19 words, with UP and DOWN averaging 11.1 and 11.8 words, respectively. The difference is small enough to be noise, the length does not carry a predictive signal.

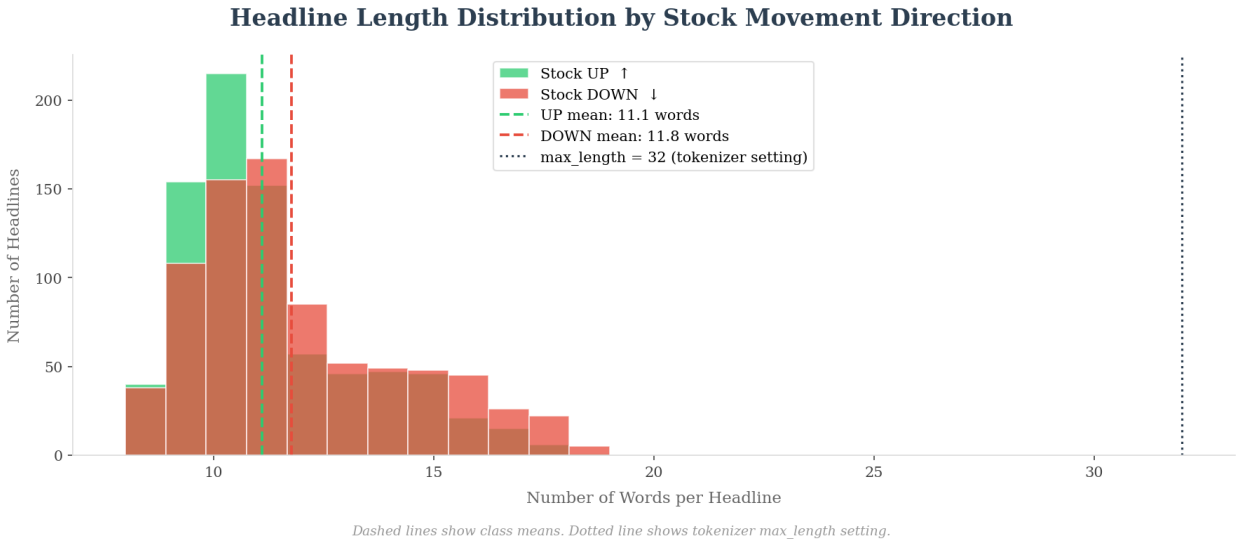

Figure 2: Headline length distribution by class. Distributions overlap almost entirely.

Data Partitioning

The dataset is split into three non-overlapping subsets using stratified sampling to preserve the 50/50 class balance in each split.

Split	Samples	Proportion	UP	DOWN
Training	1,119	69.9%	560	559
Validation	241	15.1%	120	121
Test	240	15.0%	120	120
Total	1,600	100%	800	800

Table 2: Dataset splits. The test set was not examined until final evaluation.

Text Preprocessing

Raw text cannot be fed directly into a neural network. The ALBERT tokenizer converts each headline into numerical representations through three steps.

Subword tokenization: ALBERT does not split text word by word. Complex or compound words are broken into smaller known pieces. For example:

"better-than-expected" → `[better, -, than, -, exp, ect, ed]` (7 tokens)

A headline with 8 words can produce significantly more tokens after this expansion. This is why `max_length = 32` was chosen rather than the observed word maximum of 19, it ensures no headline is truncated.

Special tokens: `[CLS]` is prepended and `[SEP]` appended to every sequence. After ALBERT processes the full sequence, the `[CLS]` token holds a contextual representation of the entire sentence, which is passed to the classification layer.

Padding: sequences shorter than `max_length` are padded with zeros, and an attention mask marks which positions contain real tokens versus padding.

Model

Architecture

The model is `AlbertForSequenceClassification` using the `albert-base-v2` pretrained checkpoint. ALBERT (A Lite BERT) reduces BERT's memory footprint through two techniques: parameter sharing across transformer layers, and factorized embedding parameterization. This keeps performance competitive while significantly reducing model size.

A two-class linear classification head is added on top of the pretrained encoder. The `[CLS]` token representation flows into this head to produce the final UP/DOWN prediction.

Property	Value
Architecture	ALBERT-base-v2
Parameters	11,685,122
Task	Binary classification
Framework	PyTorch + HuggingFace Transformers
Input length	32 tokens (max)

Table 3: Model configuration.

Training Setup

All model weights are unfrozen, this is full fine-tuning, not feature extraction. Training runs for 3 epochs using AdamW with a learning rate of 2×10^{-5} and a batch size of 16.

Hyperparameter	Value
Optimizer	AdamW
Learning rate	2×10^{-5}
Batch size	16
Epochs	3
Loss function	Cross-entropy

Table 4: Training hyperparameters.

Results

Training Behavior

Training and validation accuracy both reach 100% within the first epoch. Validation loss drops to near zero from the first batch, before training accuracy does.

This is unusual. In standard fine-tuning, training accuracy rises gradually while validation lags behind. The reversed pattern here is explained by the dataset: the synthetic headlines follow consistent templates, so the model picks up surface patterns almost immediately. There is no meaningful generalization challenge when the test distribution mirrors the training distribution exactly.

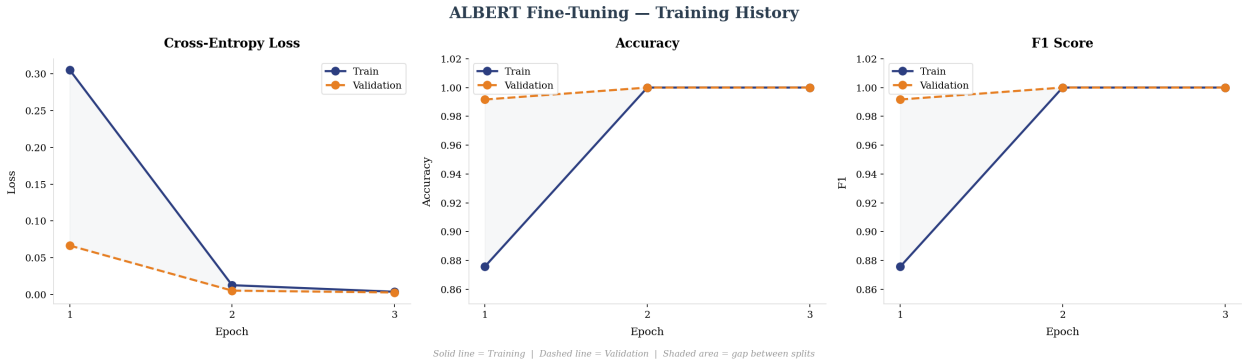

Figure 3: Training history across 3 epochs. Cross-entropy loss, accuracy, and F1 score.

Test Set Performance

Metric	Score
Accuracy	100%
F1 Score	100%

Table 5: Final results on the held-out test set.

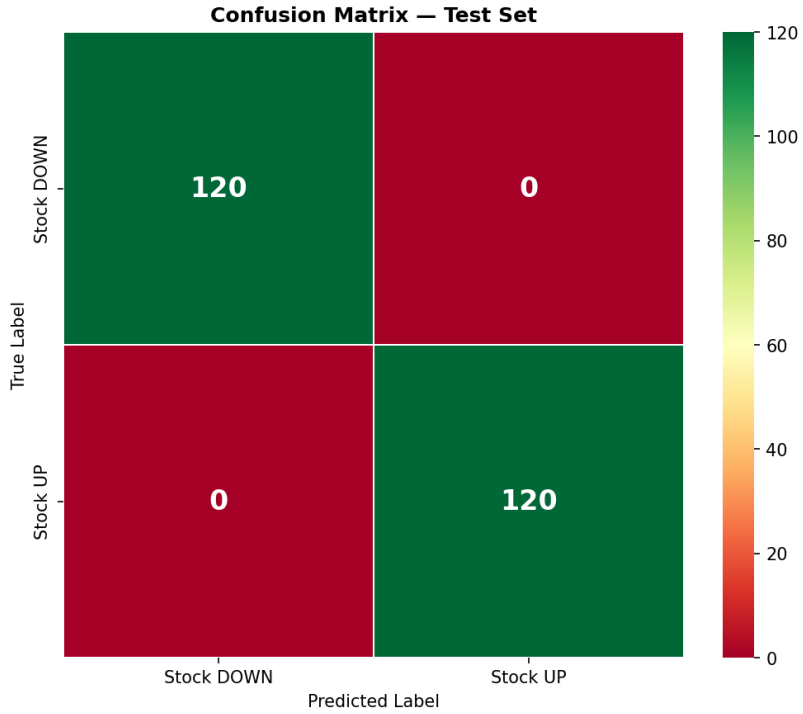

Figure 4: Confusion matrix on the test set. 240 correct predictions, 0 errors.

The model classifies all 240 test headlines correctly. This result is expected: the test set was drawn from the same synthetic distribution as the training data.

Behavioral Probing

Test set metrics confirm the model works on this data. They say nothing about what the model actually learned.

To probe that, the model is tested on plain English headlines that never appeared in training, no financial template structure, no domain specific phrasing.

Headline	Prediction	Confidence	Correct?
"Google improves in the last years"	UP	99.9%	Yes
"Microsoft increase their performance"	UP	99.9%	Yes
"Pfizer decrease their performance"	DOWN	99.9%	Yes
"Apple goes down"	UP	99.8%	**No**
"AMD starts asking what is happening"	UP	99.9%	**No**
"Meta has a massive layoff"	UP	99.9%	**No**

Table 6: Behavioral probing results on out-of-distribution headlines.

Failure Mode 1: Out-of-Distribution Language

The headline *"Apple goes down"* is predicted UP with 99.8% confidence. The training data used specific financial phrasing for negative movement: *sending shares lower*, *hurting investor sentiment*,

raising concerns across the sector. The phrase *goes down* never appeared in that context. When the model encounters language outside its training distribution, it defaults to a confident wrong prediction with no mechanism to flag uncertainty.

Failure Mode 2: Overconfidence on Ambiguous Input

The headline *"AMD starts asking what is happening"* carries no financial signal whatsoever, yet the model assigns 99.9% confidence to UP. A well-calibrated model would output something close to 50/50 on genuinely ambiguous inputs. Instead, ALBERT commits fully to a class. This overconfidence on ambiguous inputs is a known problem in deep learning and an active area of research: model calibration.

These failures are not bugs. They are the most informative part of this project. They show exactly what the model learned: financial template patterns. And exactly what it did not: general language understanding.

Limitations and Next Steps

Limitations

1. **Synthetic dataset.** Templated language makes the classification task too easy. The model learns surface patterns, not financial reasoning.
2. **No temporal validation.** In real finance, models should be tested on future data, not random splits from the same period. Random splitting leaks temporal information.
3. **Single headline.** Stock prices respond to many simultaneous signals. One headline is rarely sufficient for reliable prediction.
4. **No calibration.** The model is systematically overconfident, assigning near-certain probabilities even on ambiguous or out-of-distribution inputs.

Conclusion

This project built a complete NLP classification pipeline, from exploratory analysis through behavioral probing, using ALBERT fine-tuned on financial headlines. The model achieves perfect metrics on the synthetic test set, which tells us it learned what it was trained on. The behavioral probing tells us the more important thing: what it did not learn, and why that matters.

The gap between a model that works on templated data and a model that works on real language is exactly the gap this project makes visible. Closing that gap, with real data, temporal splits, and calibration, is where the interesting work begins.